CHAPTER

12 Entity–Relationship Modeling

Chapter Objectives

In this chapter you will learn:

- How to use Entity–Relationship (ER) modeling in database design.
- The basic concepts associated with the ER model: entities, relationships, and attributes.
- A diagrammatic technique for displaying an ER model using the Unified Modeling Language (UML).
- How to identify and resolve problems with ER models called connection traps.

In Chapter 11 we described the main techniques for gathering and capturing information about what the users require of a database system. Once the requirements collection and analysis stage of the database system development lifecycle is complete and we have documented the requirements for the database system, we are ready to begin the database design stage.

One of the most difficult aspects of database design is the fact that designers, programmers, and end-users tend to view data and its use in different ways. Unfortunately, unless we gain a common understanding that reflects how the enterprise operates, the design we produce will fail to meet the users' requirements. To ensure that we get a precise understanding of the nature of the data and how it is used by the enterprise, we need a model for communication that is nontechnical and free of ambiguities. The Entity–Relationship (ER) model is one such example. ER modeling is a top-down approach to database design that begins by identifying the important data called *entities* and *relationships* between the data that must be represented in the model. We then add more details, such as the information we want to hold about the entities and relationships called *attributes* and any *constraints* on the entities, relationships, and attributes. ER modeling is an important technique for any database designer to master and forms the basis of the methodology presented in this book.

In this chapter we introduce the basic concepts of the ER model. Although there is general agreement about what each concept means, there are a number of different notations that can be used to represent each concept diagrammatically. We have chosen a diagrammatic notation that uses an increasingly popular object-oriented modeling language called the **Unified Modeling Language** (**UML**) (Booch *et al.*,

1999). UML is the successor to a number of object-oriented analysis and design methods introduced in the 1980s and 1990s. The Object Management Group (OMG) is responsible for the creation and management of UML (available at www.uml.org). UML is currently recognized as the de facto industry standard modeling language for object-oriented software engineering projects. Although we use the UML notation for drawing ER models, we continue to describe the concepts of ER models using traditional database terminology. In Section 27.8 we will provide a fuller discussion on UML. We also include a summary of two alternative diagrammatic notations for ER models in Appendix C.

In the next chapter we discuss the inherent problems associated with representing complex database applications using the basic concepts of the ER model. To overcome these problems, additional "semantic" concepts were added to the original ER model, resulting in the development of the Enhanced Entity–Relationship (EER) model. In Chapter 13 we describe the main concepts associated with the EER model, called specialization/generalization, aggregation, and composition. We also demonstrate how to convert the ER model shown in Figure 12.1 into the EER model shown in Figure 13.8.

Structure of this Chapter In Sections 12.1, 12.2, and 12.3 we introduce the basic concepts of the Entity–Relationship model: entities, relationships, and attributes. In each section we illustrate how the basic ER concepts are represented pictorially in an ER diagram using UML. In Section 12.4 we differentiate between weak and strong entities and in Section 12.5 we discuss how attributes normally associated with entities can be assigned to relationships. In Section 12.6 we describe the structural constraints associated with relationships. Finally, in Section 12.7 we identify potential problems associated with the design of an ER model called connection traps and demonstrate how these problems can be resolved.

The ER diagram shown in Figure 12.1 is an example of one of the possible end-products of ER modeling. This model represents the relationships between data described in the requirements specification for the Branch view of the *DreamHome* case study given in Appendix A. This figure is presented at the start of this chapter to show the reader an example of the type of model that we can build using ER modeling. At this stage, the reader should not be concerned about fully understanding this diagram, as the concepts and notation used in this figure are discussed in detail throughout this chapter.

12.1 Entity Types

Entity type A group of objects with the same properties, which are identified by the enterprise as having an independent existence.

The basic concept of the ER model is the **entity type**, which represents a group of "objects" in the "real world" with the same properties. An entity type has an

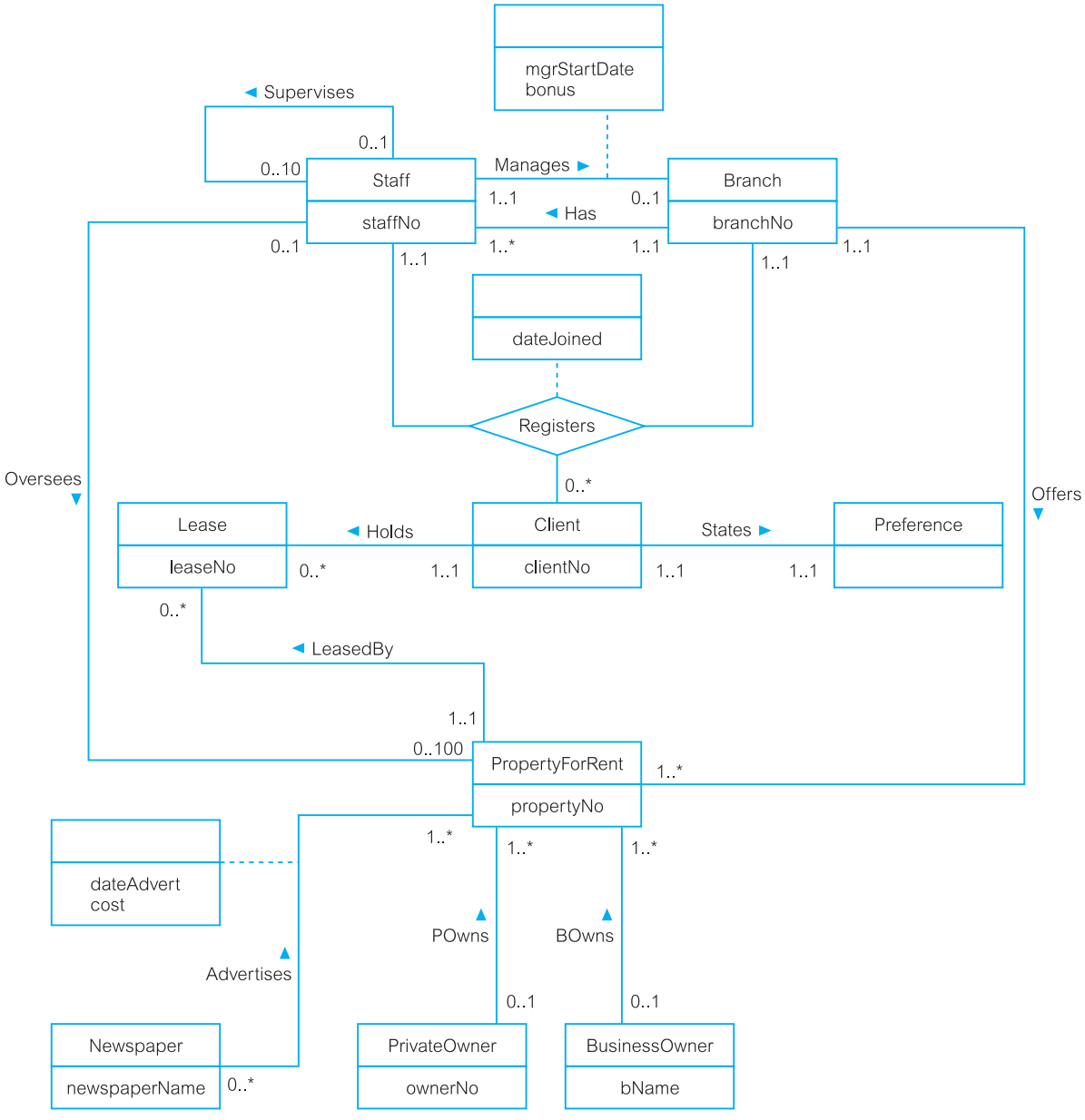

Figure 12.1 An Entity–Relationship (ER) diagram of the Branch view of *DreamHome*.

independent existence and can be objects with a physical (or "real") existence or objects with a conceptual (or "abstract") existence, as listed in Figure 12.2. Note that we are able to give only a working definition of an entity type, as no strict formal definition exists. This means that different designers may identify different entities.

Entity occurrence A uniquely identifiable object of an entity type.

Physical existence	
Staff	Part
Property	Supplier
Customer	Product
Conceptual existence	
Viewing	Sale
Inspection	Work experience

Figure 12.2 Example of entities with a physical or conceptual existence.

Each uniquely identifiable object of an entity type is referred to simply as an **entity occurrence**. Throughout this book, we use the terms "entity type" or "entity occurrence"; however, we use the more general term "entity" where the meaning is obvious.

We identify each entity type by a name and a list of properties. A database normally contains many different entity types. Examples of entity types shown in Figure 12.1 include: Staff, Branch, PropertyForRent, and PrivateOwner.

Diagrammatic representation of entity types

Each entity type is shown as a rectangle, labeled with the name of the entity, which is normally a singular noun. In UML, the first letter of each word in the entity name is uppercase (for example, Staff and PropertyForRent). Figure 12.3 illustrates the diagrammatic representation of the Staff and Branch entity types.

12.2 Relationship Types

Relationship type A set of meaningful associations among entity types.

A **relationship type** is a set of associations between one or more participating entity types. Each relationship type is given a name that describes its function. An example of a relationship type shown in Figure 12.1 is the relationship called *POwns,* which associates the PrivateOwner and PropertyForRent entities.

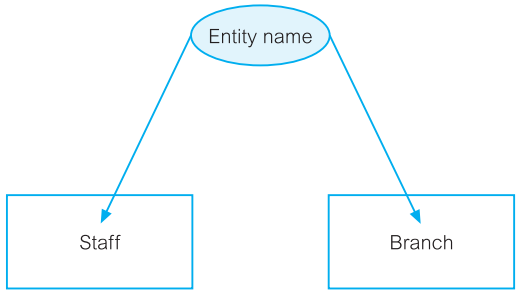

Figure 12.3 Diagrammatic representation of the Staff and Branch entity types.

As with entity types and entities, it is necessary to distinguish between the terms "relationship type" and "relationship occurrence."

Relationship occurrence	A uniquely identifiable association that includes one occurrence from each participating entity type.

A relationship occurrence indicates the particular entity occurrences that are related. Throughout this book, we use the terms "relationship type" or "relationship occurrence." However, as with the term "entity," we use the more general term "relationship" when the meaning is obvious.

Consider a relationship type called *Has*, which represents an association between Branch and Staff entities, that is Branch *Has* Staff. Each occurrence of the *Has* relationship associates one Branch entity occurrence with one Staff entity occurrence. We can examine examples of individual occurrences of the *Has* relationship using a *semantic net*. A semantic net is an object-level model, which uses the symbol • to represent entities and the symbol ◈ to represent relationships. The semantic net in Figure 12.4 shows three examples of the *Has* relationships (denoted rl, r2, and r3). Each relationship describes an association of a single Branch entity occurrence with a single Staff entity occurrence. Relationships are represented by lines that join each participating Branch entity with the associated Staff entity. For example, relationship rl represents the association between Branch entity B003 and Staff entity SG37.

Note that we represent each Branch and Staff entity occurrences using values for the primary key attributes, namely branchNo and staffNo. Primary key attributes uniquely identify each entity occurrence and are discussed in detail in the following section.

If we represented an enterprise using semantic nets, it would be difficult to understand, due to the level of detail. We can more easily represent the relationships between entities in an enterprise using the concepts of the ER model. The ER model uses a higher level of abstraction than the semantic net by combining sets of entity occurrences into entity types and sets of relationship occurrences into relationship types.

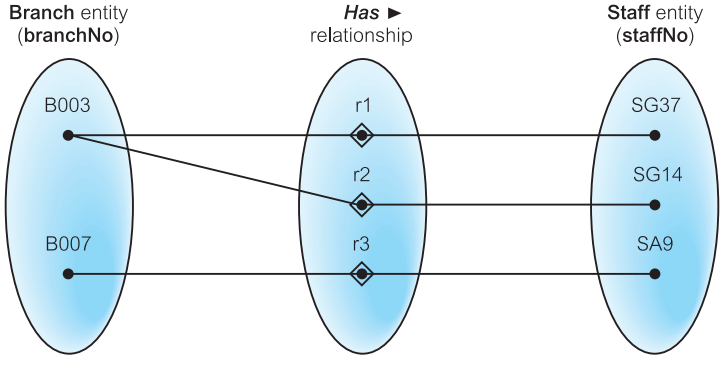

Figure 12.4 A semantic net showing individual occurrences of the *Has* relationship type.

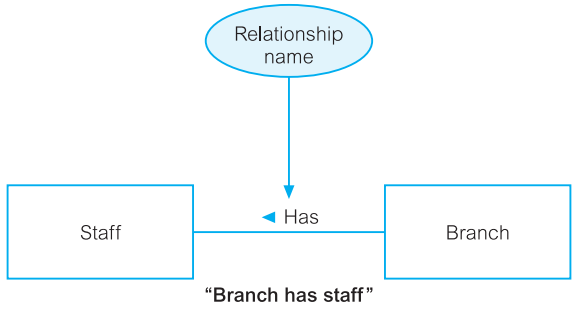

Figure 12.5
A diagrammatic representation of Branch *Has* Staff relationship type.

Diagrammatic representation of relationship types

Each relationship type is shown as a line connecting the associated entity types and labeled with the name of the relationship. Normally, a relationship is named using a verb (for example, *Supervises* or *Manages*) or a short phrase including a verb (for example, *LeasedBy*). Again, the first letter of each word in the relationship name is shown in uppercase. Whenever possible, a relationship name should be unique for a given ER model.

A relationship is only labeled in one direction, which normally means that the name of the relationship only makes sense in one direction (for example, Branch *Has* Staff makes more sense than Staff *Has* Branch). So once the relationship name is chosen, an arrow symbol is placed beside the name indicating the correct direction for a reader to interpret the relationship name (for example, Branch Has ▶ Staff) as shown in Figure 12.5.

12.2.1 Degree of Relationship Type

Degree of a relationship type — The number of participating entity types in a relationship.

The entities involved in a particular relationship type are referred to as **participants** in that relationship. The number of participants in a relationship type is called the **degree** of that relationship. Therefore, the degree of a relationship indicates the number of entity types involved in a relationship. A relationship of degree two is called **binary**. An example of a binary relationship is the *Has* relationship shown in Figure 12.5 with two participating entity types; namely, Staff and Branch. A second example of a binary relationship is the *POwns* relationship shown in Figure 12.6 with two participating entity types; namely, PrivateOwner and PropertyForRent. The *Has* and *POwns* relationships are also shown in Figure 12.1 as well as other examples of

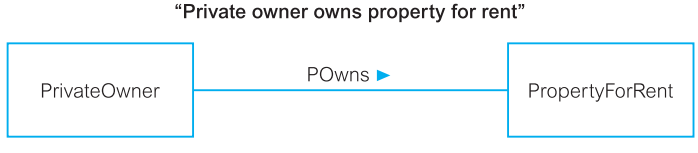

Figure 12.6
An example of a binary relationship called *POwns*.

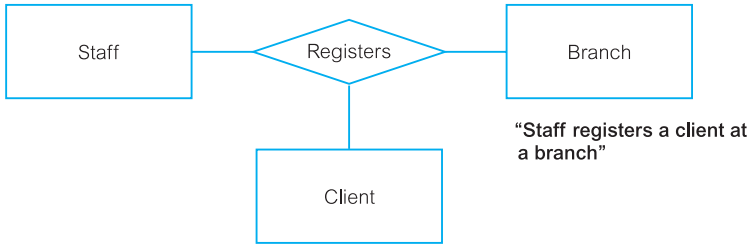

Figure 12.7 An example of a ternary relationship called *Registers*.

binary relationships. In fact, the most common degree for a relationship is binary, as demonstrated in this figure.

A relationship of degree three is called **ternary**. An example of a ternary relationship is *Registers* with three participating entity types: Staff, Branch, and Client. This relationship represents the registration of a client by a member of staff at a branch. The term "complex relationship" is used to describe relationships with degrees higher than binary.

Diagrammatic representation of complex relationships

The UML notation uses a diamond to represent relationships with degrees higher than binary. The name of the relationship is displayed inside the diamond, and in this case, the directional arrow normally associated with the name is omitted. For example, the ternary relationship called *Registers* is shown in Figure 12.7. This relationship is also shown in Figure 12.1.

A relationship of degree four is called **quaternary**. As we do not have an example of such a relationship in Figure 12.1, we describe a quaternary relationship called *Arranges* with four participating entity types—namely, Buyer, Solicitor, FinancialInstitution, and Bid—in Figure 12.8. This relationship represents the situation where a buyer, advised by a solicitor and supported by a financial institution, places a bid.

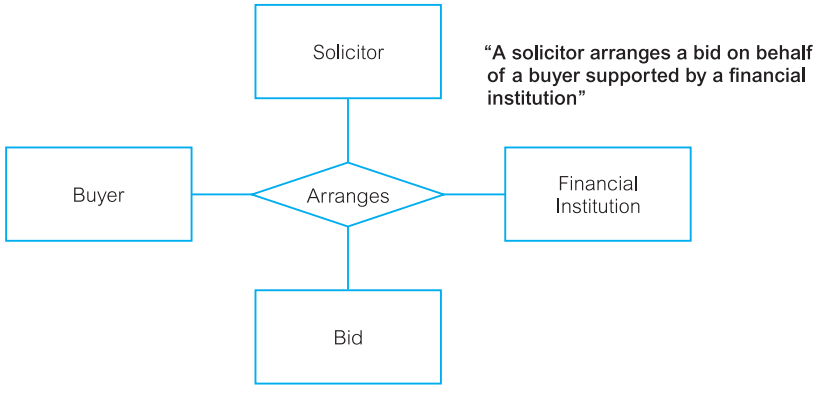

Figure 12.8 An example of a quaternary relationship called *Arranges*.

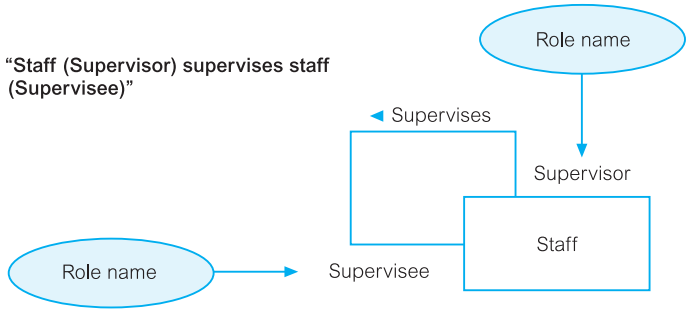

Figure 12.9
An example of a recursive relationship called *Supervises* with role names Supervisor and Supervisee.

12.2.2 Recursive Relationship

Recursive relationship — A relationship type in which the *same* entity type participates more than once in *different roles*.

Consider a recursive relationship called *Supervises*, which represents an association of staff with a Supervisor where the Supervisor is also a member of staff. In other words, the Staff entity type participates twice in the *Supervises* relationship; the first participation as a Supervisor, and the second participation as a member of staff who is supervised (Supervisee). Recursive relationships are sometimes called *unary* relationships.

Relationships may be given **role names** to indicate the purpose that each participating entity type plays in a relationship. Role names can be important for recursive relationships to determine the function of each participant. The use of role names to describe the *Supervises* recursive relationship is shown in Figure 12.9. The first participation of the Staff entity type in the *Supervises* relationship is given the role name "Supervisor" and the second participation is given the role name "Supervisee."

Role names may also be used when two entities are associated through more than one relationship. For example, the Staff and Branch entity types are associated through two distinct relationships called *Manages* and *Has*. As shown in Figure 12.10,

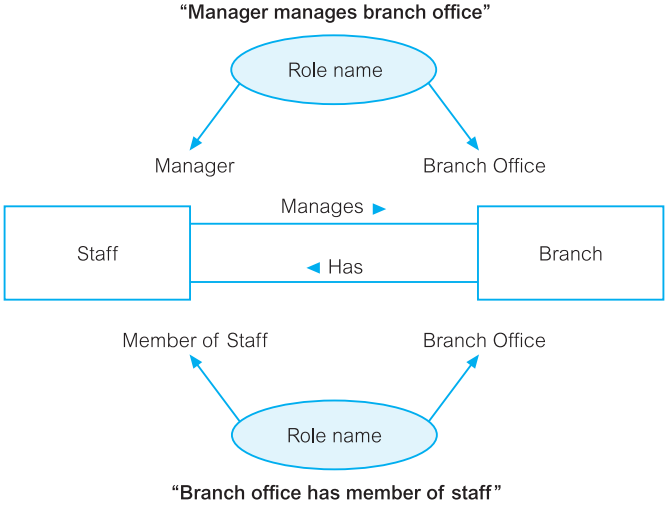

Figure 12.10
An example of entities associated through two distinct relationships called *Manages* and *Has* with role names.

the use of role names clarifies the purpose of each relationship. For example, in the case of Staff *Manages* Branch, a member of staff (Staff entity) given the role name "Manager" manages a branch (Branch entity) given the role name "Branch Office." Similarly, for Branch *Has* Staff, a branch, given the role name "Branch Office" has staff given the role name "Member of Staff."

Role names are usually not required if the function of the participating entities in a relationship is unambiguous.

12.3 Attributes

Attribute A property of an entity or a relationship type.

The particular properties of entity types are called attributes. For example, a Staff entity type may be described by the staffNo, name, position, and salary attributes. The attributes hold values that describe each entity occurrence and represent the main part of the data stored in the database.

A relationship type that associates entities can also have attributes similar to those of an entity type, but we defer discussion of this until Section 12.5. In this section, we concentrate on the general characteristics of attributes.

Attribute domain The set of allowable values for one or more attributes.

Each attribute is associated with a set of values called a **domain**. The domain defines the potential values that an attribute may hold and is similar to the domain concept in the relational model (see Section 4.2). For example, the number of rooms associated with a property is between 1 and 15 for each entity occurrence. We therefore define the set of values for the number of rooms (rooms) attribute of the PropertyForRent entity type as the set of integers between 1 and 15.

Attributes may share a domain. For example, the address attributes of the Branch, PrivateOwner, and BusinessOwner entity types share the same domain of all possible addresses. Domains can also be composed of domains. For example, the domain for the address attribute of the Branch entity is made up of subdomains: street, city, and postcode.

The domain of the name attribute is more difficult to define, as it consists of all possible names. It is certainly a character string, but it might consist not only of letters but also hyphens or other special characters. A fully developed data model includes the domains of each attribute in the ER model.

As we now explain, attributes can be classified as being: *simple* or *composite, single-valued* or *multi-valued,* or *derived.*

12.3.1 Simple and Composite Attributes

Simple attribute An attribute composed of a single component with an independent existence.

Simple attributes cannot be further subdivided into smaller components. Examples of simple attributes include position and salary of the Staff entity. Simple attributes are sometimes called *atomic* attributes.

Composite attribute An attribute composed of multiple components, each with an independent existence.

Some attributes can be further divided to yield smaller components with an independent existence of their own. For example, the address attribute of the Branch entity with the value (163 Main St, Glasgow, G11 9QX) can be subdivided into street (163 Main St), city (Glasgow), and postcode (G11 9QX) attributes.

The decision to model the address attribute as a simple attribute or to subdivide the attribute into street, city, and postcode is dependent on whether the user view of the data refers to the address attribute as a single unit or as individual components.

12.3.2 Single-valued and Multi-valued Attributes

Single-valued attribute An attribute that holds a single value for each occurrence of an entity type.

The majority of attributes are single-valued. For example, each occurrence of the Branch entity type has a single value for the branch number (branchNo) attribute (for example, B003), and therefore the branchNo attribute is referred to as being single-valued.

Multi-valued attribute An attribute that holds multiple values for each occurrence of an entity type.

Some attributes have multiple values for each entity occurrence. For example, each occurrence of the Branch entity type can have multiple values for the telNo attribute (for example, branch number B003 has telephone numbers 0141-339-2178 and 0141-339-4439) and therefore the telNo attribute in this case is multi-valued. A multi-valued attribute may have a set of numbers with upper and lower limits. For example, the telNo attribute of the Branch entity type has between one and three values. In other words, a branch may have a minimum of a single telephone number to a maximum of three telephone numbers.

12.3.3 Derived Attributes

Derived attribute An attribute that represents a value that is derivable from the value of a related attribute or set of attributes, not necessarily in the same entity type.

The values held by some attributes may be derived. For example, the value for the duration attribute of the Lease entity is calculated from the rentStart and rentFinish attributes, also of the Lease entity type. We refer to the duration attribute as a derived attribute, the value of which is derived from the rentStart and rentFinish attributes.

In some cases, the value of an attribute is derived from the entity occurrences in the same entity type. For example, the total number of staff (totalStaff) attribute of the Staff entity type can be calculated by counting the total number of Staff entity occurrences.

Derived attributes may also involve the association of attributes of different entity types. For example, consider an attribute called deposit of the Lease entity type. The value of the deposit attribute is calculated as twice the monthly rent for a property. Therefore, the value of the deposit attribute of the Lease entity type is derived from the rent attribute of the PropertyForRent entity type.

12.3.4 Keys

Candidate key — The minimal set of attributes that uniquely identifies each occurrence of an entity type.

A candidate key is the minimal number of attributes, whose value(s) uniquely identify each entity occurrence. For example, the branch number (branchNo) attribute is the candidate key for the Branch entity type, and has a distinct value for each branch entity occurrence. The candidate key must hold values that are unique for every occurrence of an entity type. This implies that a candidate key cannot contain a null (see Section 4.2). For example, each branch has a unique branch number (for example, B003), and there will never be more than one branch with the same branch number.

Primary key — The candidate key that is selected to uniquely identify each occurrence of an entity type.

An entity type may have more than one candidate key. For the purposes of discussion, consider that a member of staff has a unique company-defined staff number (staffNo) and also a unique National Insurance Number (NIN) that is used by the government. We therefore have two candidate keys for the Staff entity, one of which must be selected as the primary key.

The choice of primary key for an entity is based on considerations of attribute length, the minimal number of attributes required, and the future certainty of uniqueness. For example, the company-defined staff number contains a maximum of five characters (for example, SG14), and the NIN contains a maximum of nine characters (for example, WL220658D). Therefore, we select staffNo as the primary key of the Staff entity type and NIN is then referred to as the **alternate key**.

Composite key A candidate key that consists of two or more attributes.

In some cases, the key of an entity type is composed of several attributes whose values together are unique for each entity occurrence but not separately. For example, consider an entity called Advert with propertyNo (property number), newspaperName, dateAdvert, and cost attributes. Many properties are advertised in many newspapers on a given date. To uniquely identify each occurrence of the Advert entity type requires values for the propertyNo, newspaperName, and dateAdvert attributes. Thus, the Advert entity type has a composite primary key made up of the propertyNo, newspaperName, and dateAdvert attributes.

Diagrammatic representation of attributes

If an entity type is to be displayed with its attributes, we divide the rectangle representing the entity in two. The upper part of the rectangle displays the name of the entity and the lower part lists the names of the attributes. For example, Figure 12.11 shows the ER diagram for the Staff and Branch entity types and their associated attributes.

The first attribute(s) to be listed is the primary key for the entity type, if known. The name(s) of the primary key attribute(s) can be labeled with the tag {PK}. In UML, the name of an attribute is displayed with the first letter in lowercase and, if the name has more than one word, with the first letter of each subsequent word in uppercase (for example, address and telNo). Additional tags that can be used include partial primary key {PPK} when an attribute forms part of a composite primary key, and alternate key {AK}. As shown in Figure 12.11, the primary key of the Staff entity type is the staffNo attribute and the primary key of the Branch entity type is the branchNo attribute.

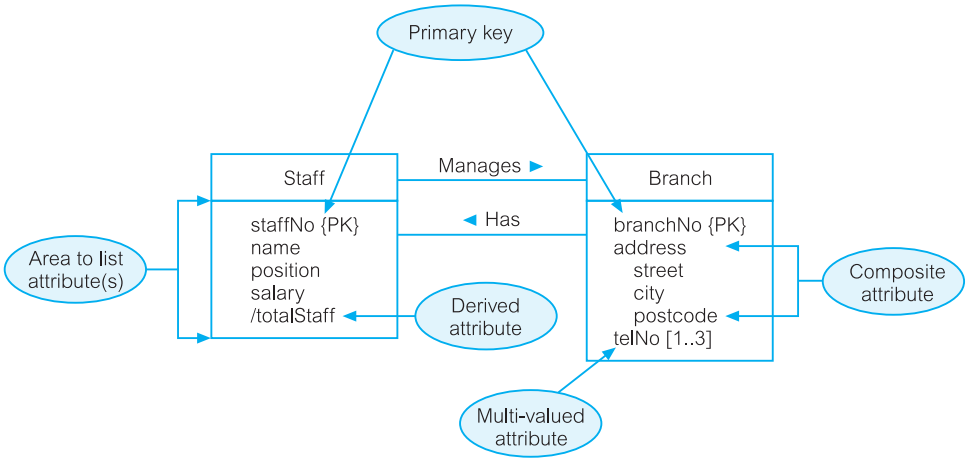

Figure 12.11 Diagrammatic representation of Staff and Branch entities and their attributes.

For some simpler database systems, it is possible to show all the attributes for each entity type in the ER diagram. However, for more complex database systems, we just display the attribute, or attributes, that form the primary key of each entity type. When only the primary key attributes are shown in the ER diagram, we can omit the {PK} tag.

For simple, single-valued attributes, there is no need to use tags, so we simply display the attribute names in a list below the entity name. For composite attributes, we list the name of the composite attribute followed below and indented to the right by the names of its simple component attributes. For example, in Figure 12.11 the composite attribute address of the Branch entity is shown, followed below by the names of its component attributes, street, city, and postcode. For multivalued attributes, we label the attribute name with an indication of the range of values available for the attribute. For example, if we label the telNo attribute with the range [1..*], this means that the values for the telNo attribute is one or more. If we know the precise maximum number of values, we can label the attribute with an exact range. For example, if the telNo attribute holds one to a maximum of three values, we can label the attribute with [1..3].

For derived attributes, we prefix the attribute name with a "/". For example, the derived attribute of the Staff entity type is shown in Figure 12.11 as /totalStaff.

12.4 Strong and Weak Entity Types

We can classify entity types as being strong or weak.

Strong entity type An entity type that is *not* existence-dependent on some other entity type.

An entity type is referred to as being strong if its existence does not depend upon the existence of another entity type. Examples of strong entities are shown in Figure 12.1 and include the Staff, Branch, PropertyForRent, and Client entities. A characteristic of a strong entity type is that each entity occurrence is uniquely identifiable using the primary key attribute(s) of that entity type. For example, we can uniquely identify each member of staff using the staffNo attribute, which is the primary key for the Staff entity type.

Weak entity type An entity type that is existence-dependent on some other entity type.

A weak entity type is dependent on the existence of another entity type. An example of a weak entity type called Preference is shown in Figure 12.12. A characteristic of a weak entity is that each entity occurrence cannot be uniquely identified using only the attributes associated with that entity type. For example, note that there is no primary key for the Preference entity. This means that we cannot identify each occurrence of the Preference entity type using only the attributes of this

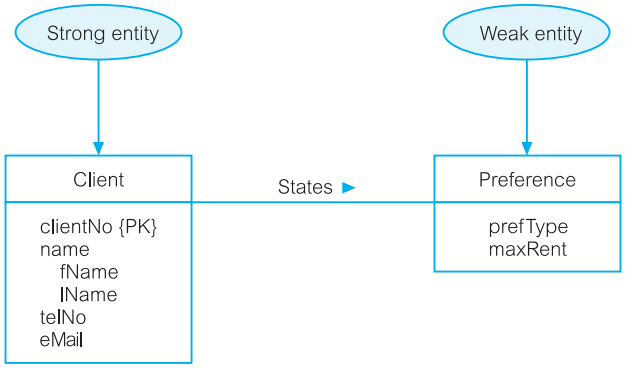

Figure 12.12 A strong entity type called Client and a weak entity type called Preference.

entity. We can uniquely identify each preference only through the relationship that a preference has with a client who is uniquely identifiable using the primary key for the Client entity type, namely clientNo. In this example, the Preference entity is described as having existence dependency for the Client entity, which is referred to as being the owner entity.

Weak entity types are sometimes referred to as *child, dependent,* or *subordinate* entities and strong entity types as *parent, owner,* or *dominant* entities.

12.5 Attributes on Relationships

As we mentioned in Section 12.3, attributes can also be assigned to relationships. For example, consider the relationship *Advertises*, which associates the Newspaper and PropertyForRent entity types as shown in Figure 12.1. To record the date the property was advertised and the cost, we associate this information with the *Advertises* relationship as attributes called dateAdvert and cost, rather than with the Newspaper or the PropertyForRent entities.

Diagrammatic representation of attributes on relationships

We represent attributes associated with a relationship type using the same symbol as an entity type. However, to distinguish between a relationship with an attribute and an entity, the rectangle representing the attribute(s) is associated with the relationship using a dashed line. For example, Figure 12.13 shows the *Advertises* relationship with the attributes dateAdvert and cost. A second example shown in Figure 12.1 is the *Manages* relationship with the mgrStartDate and bonus attributes.

The presence of one or more attributes assigned to a relationship may indicate that the relationship conceals an unidentified entity type. For example, the presence of the dateAdvert and cost attributes on the *Advertises* relationship indicates the presence of an entity called Advert.

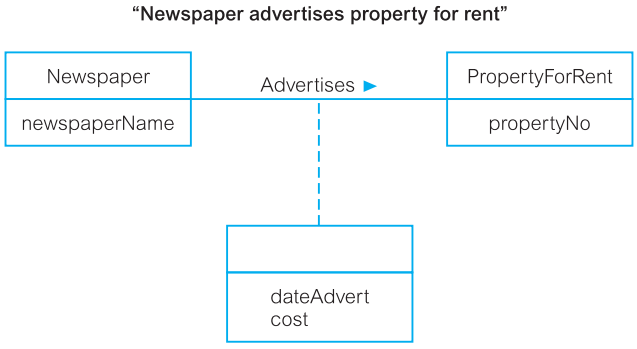

Figure 12.13 An example of a relationship called *Advertises* with attributes dateAdvert and cost.

12.6 Structural Constraints

We now examine the constraints that may be placed on entity types that participate in a relationship. The constraints should reflect the restrictions on the relationships as perceived in the "real world." Examples of such constraints include the requirements that a property for rent must have an owner and each branch must have staff. The main type of constraint on relationships is called **multiplicity**.

Multiplicity The number (or range) of possible occurrences of an entity type that may relate to a single occurrence of an associated entity type through a particular relationship.

Multiplicity constrains the way that entities are related. It is a representation of the policies (or business rules) established by the user or enterprise. Ensuring that all appropriate **constraints** are identified and represented is an important part of modeling an enterprise.

As we mentioned earlier, the most common degree for relationships is binary. Binary relationships are generally referred to as being one-to-one (1:1), one-to-many (1:*), or many-to-many (*:*). We examine these three types of relationships using the following integrity constraints:

- a member of staff manages a branch (1:1);
- a member of staff oversees properties for rent (1:*);
- newspapers advertise properties for rent (*:*).

In Sections 12.6.1, 12.6.2, and 12.6.3 we demonstrate how to determine the multiplicity for each of these constraints and show how to represent each in an ER diagram. In Section 12.6.4 we examine multiplicity for relationships of degrees higher than binary.

It is important to note that not all integrity constraints can be easily represented in an ER model. For example, the requirement that a member of staff receives an additional day's holiday for every year of employment with the enterprise may be difficult to represent in an ER model.

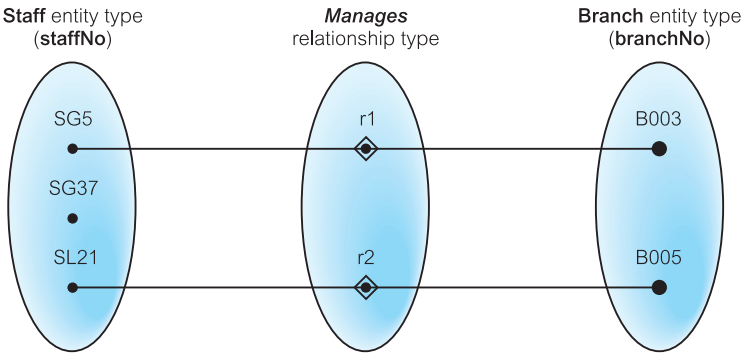

Figure 12.14(a) Semantic net showing two occurrences of the Staff *Manages* Branch relationship type.

12.6.1 One-to-One (1:1) Relationships

Consider the relationship *Manages*, which relates the Staff and Branch entity types. Figure 12.14(a) displays two occurrences of the *Manages* relationship type (denoted rl and r2) using a semantic net. Each relationship (r*n*) represents the association between a single Staff entity occurrence and a single Branch entity occurrence. We represent each entity occurrence using the values for the primary key attributes of the Staff and Branch entities, namely staffNo and branchNo.

Determining the multiplicity

Determining the multiplicity normally requires examining the precise relationships between the data given in a enterprise constraint using sample data. The sample data may be obtained by examining filled-in forms or reports and, if possible, from discussion with users. However, it is important to stress that to reach the right conclusions about a constraint requires that the sample data examined or discussed is a true representation of all the data being modeled.

In Figure 12.14(a) we see that staffNo SG5 manages branchNo B003 and staffNo SL21 manages branchNo BOO5, but staffNo SG37 does not manage any branch. In other words, a member of staff can manage *zero or one* branch and each branch is managed by *one* member of staff. As there is a maximum of one branch for each member of staff involved in this relationship and a maximum of one member of staff for each branch, we refer to this type of relationship as *one-to-one*, which we usually abbreviate as (1:1).

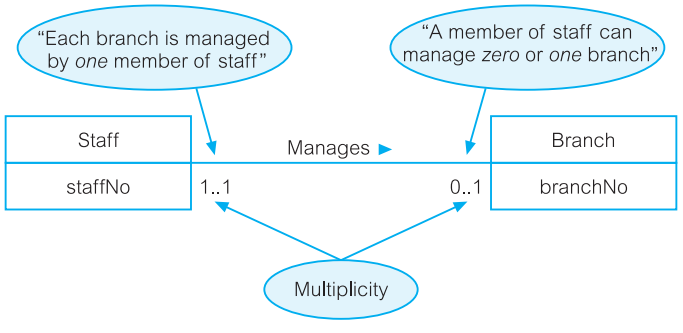

Figure 12.14(b) The multiplicity of the Staff *Manages* Branch one-to-one (1:1) relationship.

Diagrammatic representation of 1:1 relationships

An ER diagram of the Staff *Manages* Branch relationship is shown in Figure 12.14(b). To represent that a member of staff can manage *zero or one* branch, we place a "0..1" beside the Branch entity. To represent that a branch always has *one* manager, we place a "1..1" beside the Staff entity. (Note that for a 1:1 relationship, we may choose a relationship name that makes sense in either direction.)

12.6.2 One-to-Many (1:*) Relationships

Consider the relationship *Oversees*, which relates the Staff and PropertyForRent entity types. Figure 12.15(a) displays three occurrences of the Staff *Oversees* PropertyForRent relationship type (denoted rl, r2, and r3) using a semantic net. Each relationship (r*n*) represents the association between a single Staff entity occurrence and a single PropertyForRent entity occurrence. We represent each entity occurrence using the values for the primary key attributes of the Staff and PropertyForRent entities, namely staffNo and propertyNo.

Determining the multiplicity

In Figure 12.15(a) we see that staffNo SG37 oversees propertyNos PG21 and PG36, and staffNo SA9 oversees propertyNo PA14 but staffNo SG5 does not oversee any properties for rent and propertyNo PG4 is not overseen by any member of staff. In summary, a member of staff can oversee *zero or more* properties for rent and a property for rent is overseen by *zero or one* member of staff. Therefore, for members of staff participating in this relationship there are *many* properties for rent, and for properties participating in this relationship there is a maximum of *one* member of staff. We refer to this type of relationship as *one-to-many*, which we usually abbreviate as (1:*).

Diagrammatic representation of 1:* relationships An ER diagram of the Staff *Oversees* PropertyForRent relationship is shown in Figure 12.15(b). To represent that a member of staff can oversee *zero or more* properties for rent, we place a "0..*" beside the PropertyForRent entity. To represent that each property for rent is overseen by *zero or one* member of staff, we place a "0..1" beside the Staff entity. (Note that with 1:* relationships, we choose a relationship name that makes sense in the 1:* direction.)

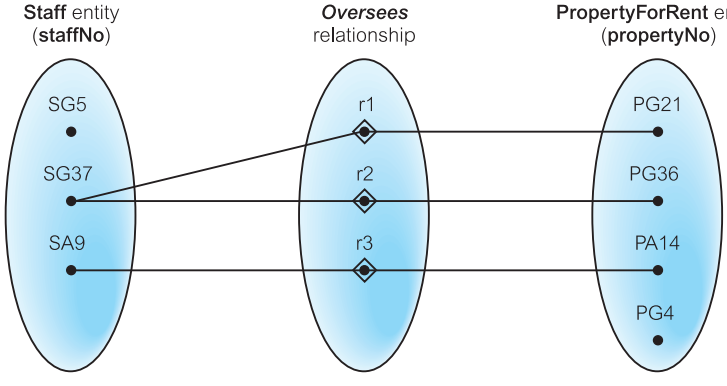

Figure 12.15(a) Semantic net showing three occurrences of the Staff *Oversees* PropertyForRent relationship type.

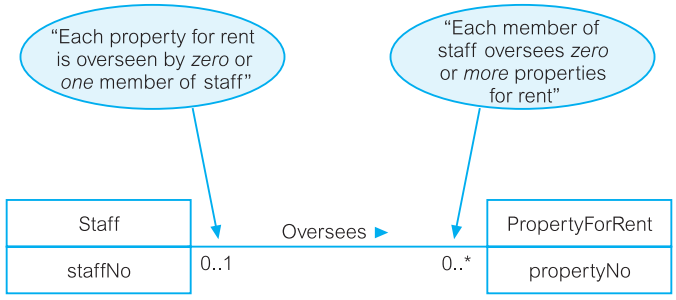

Figure 12.15(b) The multiplicity of the Staff *Oversees* PropertyForRent one-to-many (1:*) relationship type.

If we know the actual minimum and maximum values for the multiplicity, we can display these instead. For example, if a member of staff oversees a minimum of zero and a maximum of 100 properties for rent, we can replace the "0..*" with "0..100."

12.6.3 Many-to-Many (*:*) Relationships

Consider the relationship *Advertises*, which relates the Newspaper and PropertyForRent entity types. Figure 12.16(a) displays four occurrences of the *Advertises* relationship (denoted rl, r2, r3, and r4) using a semantic net. Each relationship (r*n*) represents the association between a single Newspaper entity occurrence and a single PropertyForRent entity occurrence. We represent each entity occurrence using the values for the primary key attributes of the Newspaper and PropertyForRent entity types, namely newspaperName and propertyNo.

Determining the multiplicity

In Figure 12.16(a) we see that the *Glasgow Daily* advertises propertyNos PG21 and PG36, *The West News* also advertises propertyNo PG36 and the *Aberdeen Express* advertises propertyNo PA14. However, propertyNo PG4 is not advertised in any newspaper. In other words, *one* newspaper advertises *one or more* properties for rent and *one* property for rent is advertised in *zero or more* newspapers. Therefore, for

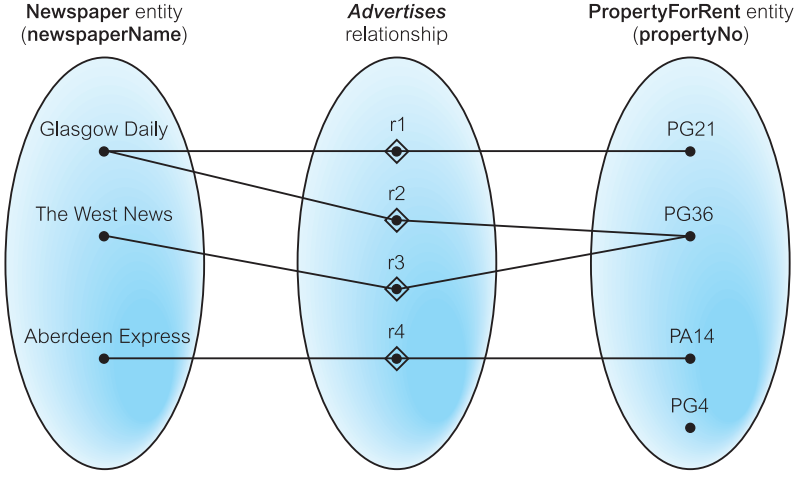

Figure 12.16(a) Semantic net showing four occurrences of the Newspaper *Advertises* PropertyForRent relationship type.

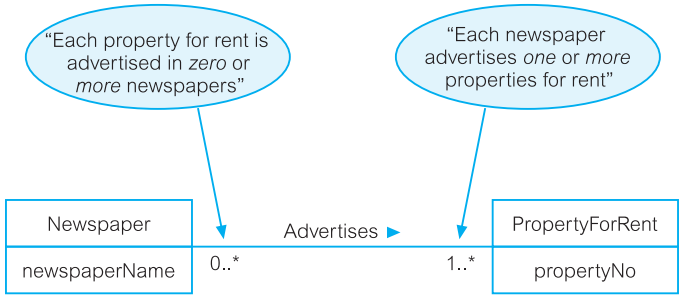

Figure 12.16(b) The multiplicity of the Newspaper *Advertises* PropertyForRent many-to-many (*:*) relationship.

newspapers there are *many* properties for rent, and for each property for rent participating in this relationship there are *many* newspapers. We refer to this type of relationship as many-to-many, which we usually abbreviate as (*:*).

Diagrammatic representation of *:* relationships

An ER diagram of the Newspaper *Advertises* PropertyForRent relationship is shown in Figure 12.16(b). To represent that each newspaper can advertise *one or more* properties for rent, we place a "1..*" beside the PropertyForRent entity type. To represent that each property for rent can be advertised by *zero or more* newspapers, we place a "0..*" beside the Newspaper entity. (Note that for a *:* relationship, we may choose a relationship name that makes sense in either direction.)

12.6.4 Multiplicity for Complex Relationships

Multiplicity for complex relationships—that is, those higher than binary—is slightly more complex.

Multiplicity (complex relationship) The number (or range) of possible occurrences of an entity type in an n-ary relationship when the other $(n–1)$ values are fixed.

In general, the multiplicity for n-ary relationships represents the potential number of entity occurrences in the relationship when $(n–1)$ values are fixed for the other participating entity types. For example, the multiplicity for a ternary relationship represents the potential range of entity occurrences of a particular entity in the relationship when the other two values representing the other two entities are fixed. Consider the ternary *Registers* relationship between Staff, Branch, and Client shown in Figure 12.7. Figure 12.17(a) displays five occurrences of the *Registers* relationship (denoted rl to r5) using a semantic net. Each relationship (rn) represents the association of a single Staff entity occurrence, a single Branch entity occurrence, and a single Client entity occurrence. We represent each entity occurrence using the values for the primary key attributes of the Staff, Branch, and Client entities, namely staffNo, branchNo, and clientNo. In Figure 12.17(a) we examine the *Registers* relationship when the values for the Staff and Branch entities are fixed.

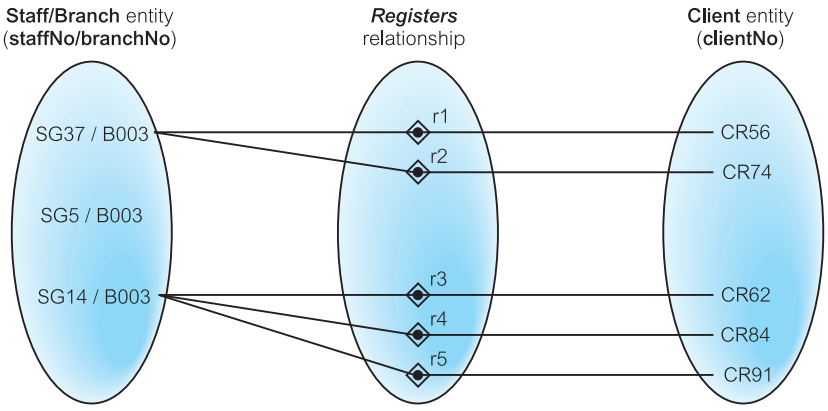

Figure 12.17(a) Semantic net showing five occurrences of the ternary *Registers* relationship with values for Staff and Branch entity types fixed.

Determining the multiplicity

In Figure 12.17(a) with the staffNo/branchNo values fixed there are *zero or more* clientNo values. For example, staffNo SG37 at branchNo B003 registers clientNo CR56 and CR74, and staffNo SG14 at branchNo B003 registers clientNo CR62, CR84, and CR91. However, SG5 at branchNo B003 registers no clients. In other words, when the staffNo and branchNo values are fixed the corresponding clientNo values are *zero or more*. Therefore, the multiplicity of the *Registers* relationship from the perspective of the Staff and Branch entities is 0..*, which is represented in the ER diagram by placing the 0..* beside the Client entity.

If we repeat this test we find that the multiplicity when Staff/Client values are fixed is 1..1, which is placed beside the Branch entity, and the Client/Branch values are fixed is 1..1, which is placed beside the Staff entity. An ER diagram of the ternary *Registers* relationship showing multiplicity is in Figure 12.17(b).

A summary of the possible ways that multiplicity constraints can be represented along with a description of the meaning is shown in Table 12.1.

12.6.5 Cardinality and Participation Constraints

Multiplicity actually consists of two separate constraints known as cardinality and participation.

Cardinality Describes the maximum number of possible relationship occurrences for an entity participating in a given relationship type.

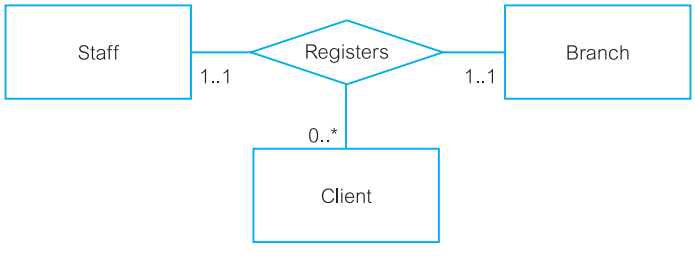

Figure 12.17(b) The multiplicity of the ternary *Registers* relationship.

TABLE 12.1 A summary of ways to represent multiplicity constraints.

ALTERNATIVE WAYS TO REPRESENT MULTIPLICITY CONSTRAINTS	MEANING
0..1	Zero or one entity occurrence
1..1 (or just 1)	Exactly one entity occurrence
0..* (or just *)	Zero or many entity occurrences
1..*	One or many entity occurrences
5..10	Minimum of 5 up to a maximum of 10 entity occurrences
0, 3, 6–8	Zero or three or six, seven, or eight entity occurrences

The cardinality of a binary relationship is what we previously referred to as a one-to-one (1:1), one-to-many (1:*), and many-to-many (*:*). The cardinality of a relationship appears as the *maximum* values for the multiplicity ranges on either side of the relationship. For example, the *Manages* relationship shown in Figure 12.18 has a one-to-one (1:1) cardinality, and this is represented by multiplicity ranges with a maximum value of 1 on both sides of the relationship.

Participation Determines whether all or only some entity occurrences participate in a relationship.

The participation constraint represents whether all entity occurrences are involved in a particular relationship (referred to as **mandatory** participation) or only some (referred to as **optional** participation). The participation of entities in a

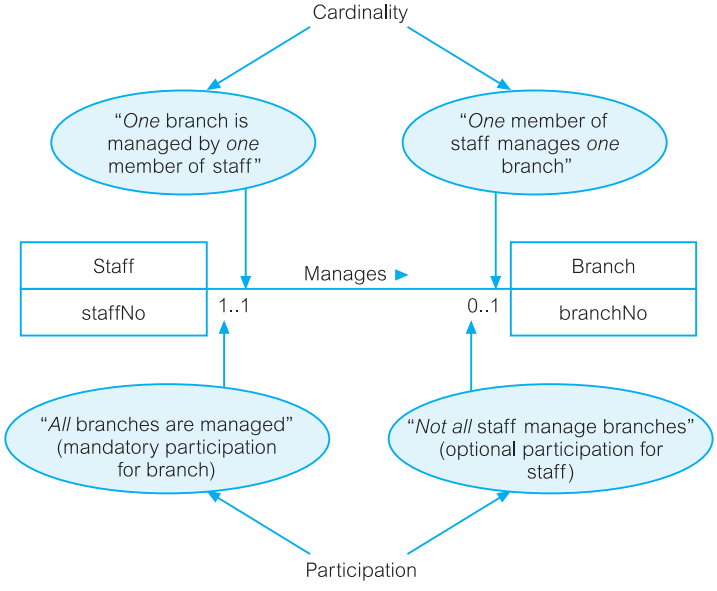

Figure 12.18 Multiplicity described as cardinality and participation constraints for the Staff *Manages* Branch (1:1) relationship.

relationship appears as the *minimum* values for the multiplicity ranges on either side of the relationship. Optional participation is represented as a minimum value of 0, and mandatory participation is shown as a minimum value of 1. It is important to note that the participation for a given entity in a relationship is represented by the minimum value on the *opposite* side of the relationship; that is, the minimum value for the multiplicity beside the related entity. For example, in Figure 12.18, the optional participation for the Staff entity in the *Manages* relationship is shown as a minimum value of 0 for the multiplicity beside the Branch entity and the mandatory participation for the Branch entity in the *Manages* relationship is shown as a minimum value of 1 for the multiplicity beside the Staff entity.

A summary of the conventions introduced in this section to represent the basic concepts of the ER model is shown on the inside front cover of this book.

12.7 Problems with ER Models

In this section we examine problems that may arise when creating an ER model. These problems are referred to as **connection traps**, and normally occur due to a misinterpretation of the meaning of certain relationships (Howe, 1989). We examine two main types of connection traps, called **fan traps** and **chasm traps**, and illustrate how to identify and resolve such problems in ER models.

In general, to identify connection traps we must ensure that the meaning of a relationship is fully understood and clearly defined. If we do not understand the relationships we may create a model that is not a true representation of the "real world."

12.7.1 Fan Traps

Fan trap Where a model represents a relationship between entity types, but the pathway between certain entity occurrences is ambiguous.

A fan trap may exist where two or more 1:* relationships fan out from the same entity. A potential fan trap is illustrated in Figure 12.19(a), which shows two 1:* relationships (*Has* and *Operates*) emanating from the same entity called Division.

This model represents the facts that a single division operates *one or more* branches and has *one or more* staff. However, a problem arises when we want to know which members of staff work at a particular branch. To appreciate the problem, we examine some occurrences of the *Has* and *Operates* relationships using values for the primary key attributes of the Staff, Division, and Branch entity types, as shown in Figure 12.19(b).

If we attempt to answer the question: "At which branch does staff number SG37 work?" we are unable to give a specific answer based on the current structure. We can determine only that staff number SG37 works at Branch B003 *or* B007. The

Figure 12.19(a) An example of a fan trap.

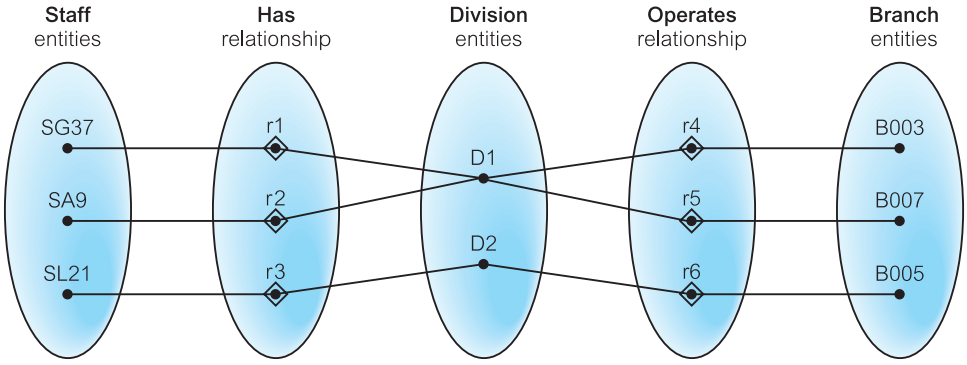

Figure 12.19(b) The semantic net of the ER model shown in Figure 12.19(a).

inability to answer this question specifically is the result of a fan trap associated with the misrepresentation of the correct relationships between the Staff, Division, and Branch entities. We resolve this fan trap by restructuring the original ER model to represent the correct association between these entities, as shown in Figure 12.20(a).

If we now examine occurrences of the *Operates* and *Has* relationships, as shown in Figure 12.20(b), we are now in a position to answer the type of question posed earlier. From this semantic net model, we can determine that staff number SG37 works at branch number B003, which is part of division Dl.

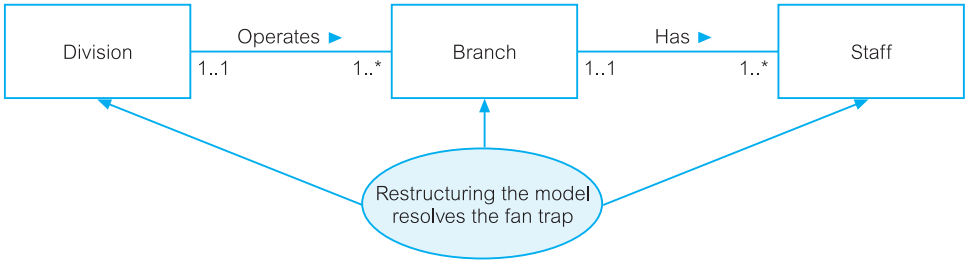

Figure 12.20(a) The ER model shown in Figure 12.19(a) restructured to remove the fan trap.

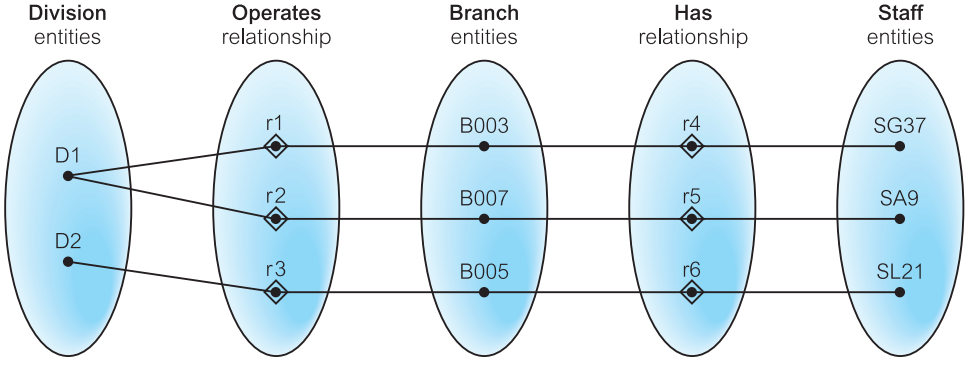

Figure 12.20(b) The semantic net of the ER model shown in Figure 12.20(a).

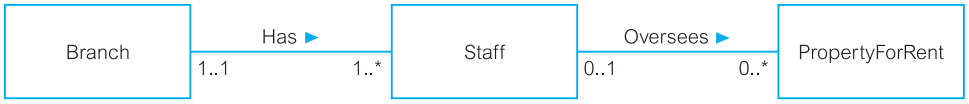

Figure 12.21(a) An example of a chasm trap.

12.7.2 Chasm Traps

Chasm trap Where a model suggests the existence of a relationship between entity types, but the pathway does not exist between certain entity occurrences.

A chasm trap may occur where there are one or more relationships with a minimum multiplicity of zero (that is, optional participation) forming part of the pathway between related entities. A potential chasm trap is illustrated in Figure 12.21(a), which shows relationships between the Branch, Staff, and PropertyForRent entities.

This model represents the facts that a single branch has *one or more* staff who oversee *zero or more* properties for rent. We also note that not all staff oversee property, and not all properties are overseen by a member of staff. A problem arises when we want to know which properties are available at each branch. To appreciate the problem, we examine some occurrences of the *Has* and *Oversees* relationships using values for the primary key attributes of the Branch, Staff, and PropertyForRent entity types, as shown in Figure 12.21(b).

If we attempt to answer the question: "At which branch is property number PA14 available?" we are unable to answer this question, as this property is not yet allocated to a member of staff working at a branch. The inability to answer this question is considered to be a loss of information (as we know a property must be available at a branch), and is the result of a chasm trap. The multiplicity of both the Staff and PropertyForRent entities in the *Oversees* relationship has a minimum value of zero, which means that some properties cannot be associated with a branch through a member of staff. Therefore, to solve this problem, we need to identify the missing relationship, which in this case is the *Offers* relationship

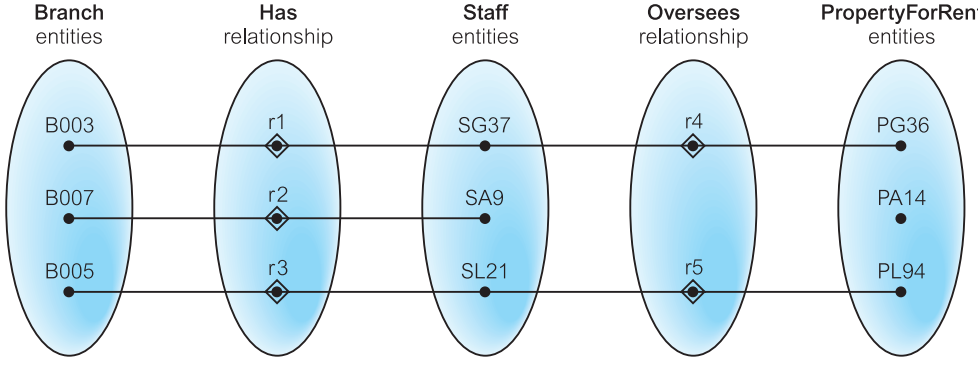

Figure 12.21(b) The semantic net of the ER model shown in Figure 12.21(a).

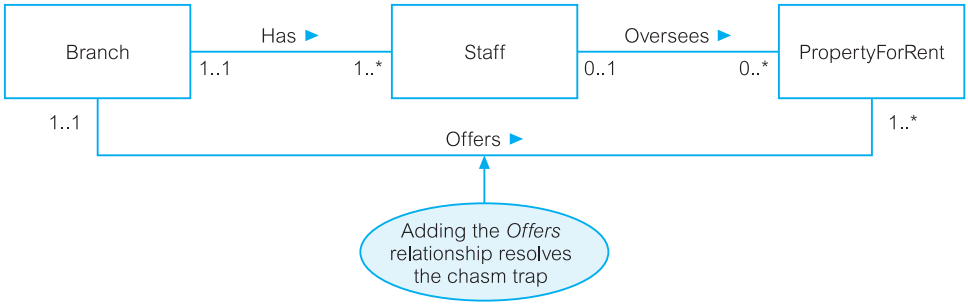

Figure 12.22(a) The ER model shown in Figure 12.21(a) restructured to remove the chasm trap.

between the Branch and PropertyForRent entities. The ER model shown in Figure 12.22(a) represents the true association between these entities. This model ensures that at all times, the properties associated with each branch are known, including properties that are not yet allocated to a member of staff.

If we now examine occurrences of the *Has*, *Oversees*, and *Offers* relationship types, as shown in Figure 12.22(b), we are now able to determine that property number PA14 is available at branch number B007.

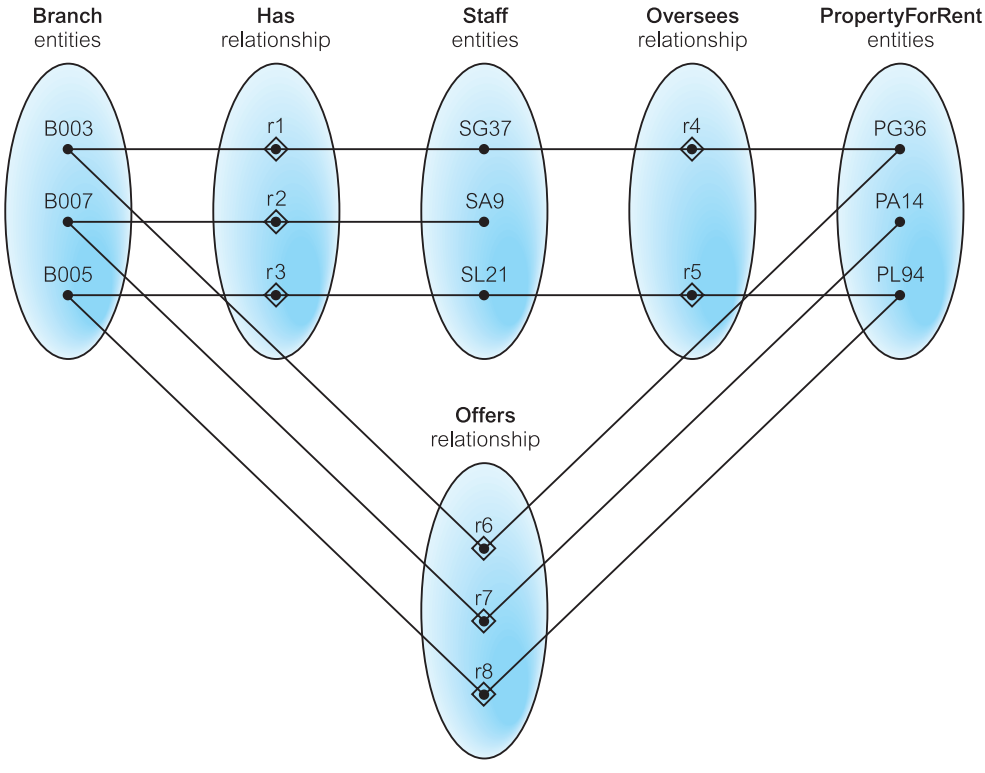

Figure 12.22(b) The semantic net of the ER model shown in Figure 12.22(a).

Chapter Summary

- An **entity type** is a group of objects with the same properties, which are identified by the enterprise as having an independent existence. An **entity occurrence** is a uniquely identifiable object of an entity type.
- A **relationship type** is a set of meaningful associations among entity types. A **relationship occurrence** is a uniquely identifiable association, which includes one occurrence from each participating entity type.
- The **degree of a relationship type** is the number of participating entity types in a relationship.
- A **recursive relationship** is a relationship type where the *same* entity type participates more than once in *different* roles.
- An **attribute** is a property of an entity or a relationship type.
- An **attribute domain** is the set of allowable values for one or more attributes.
- A **simple attribute** is composed of a single component with an independent existence.
- A **composite attribute** is composed of multiple components each with an independent existence.
- A **single-valued attribute** holds a single value for each occurrence of an entity type.
- A **multi-valued attribute** holds multiple values for each occurrence of an entity type.
- A **derived attribute** represents a value that is derivable from the value of a related attribute or set of attributes, not necessarily in the same entity.
- A **candidate key** is the minimal set of attributes that uniquely identifies each occurrence of an entity type.
- A **primary key** is the candidate key that is selected to uniquely identify each occurrence of an entity type.
- A **composite key** is a candidate key that consists of two or more attributes.
- A **strong entity type** is *not* existence-dependent on some other entity type. A **weak entity type** is existence-dependent on some other entity type.
- **Multiplicity** is the number (or range) of possible occurrences of an entity type that may relate to a single occurrence of an associated entity type through a particular relationship.
- **Multiplicity for a complex relationship** is the number (or range) of possible occurrences of an entity type in an *n*-ary relationship when the other (*n*–l) values are fixed.
- **Cardinality** describes the maximum number of possible relationship occurrences for an entity participating in a given relationship type.
- **Participation** determines whether all or only some entity occurrences participate in a given relationship.
- A **fan trap** exists where a model represents a relationship between entity types, but the pathway between certain entity occurrences is ambiguous.
- A **chasm trap** exists where a model suggests the existence of a relationship between entity types, but the pathway does not exist between certain entity occurrences.

Review Questions

12.1 Why is the ER model considered a top-down approach? Describe the four basic components of the ER model.

12.2 Describe what relationship types represent in an ER model and provide examples of unary, binary, ternary, and quaternary relationships.